`A12` Gerenciadores de Layout

O layout define a estrutura de uma interface do usuário no aplicativo, como acontece na atividade. Todos os elementos do layout são criados usando a hierarquia de objetos View e `ViewGroup`. A View geralmente desenha algo que o usuário pode ver e com que pode interagir. Já um `ViewGroup` é um contêiner invisível que define a estrutura do layout para `View` e outros objetos `ViewGroup`, como pode ser visto na Fig. 1 [1].

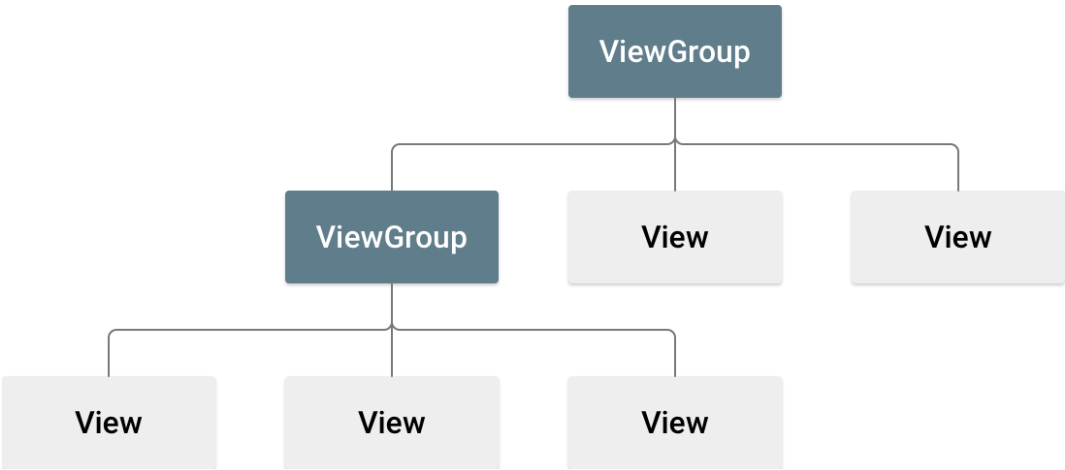

Figura 1: Hierarquia de visualização, que define o layout de uma IU.

Os objetos View geralmente são chamados de "widgets" e podem ser uma das muitas subclasses, como `Button` ou `TextView`. Os objetos `ViewGroup` geralmente são chamados de layouts e podem ser de um dos muitos tipos que fornecem uma estrutura de layout diferente, como `LinearLayout` ou `ConstraintLayout`. Um layout pode ser declarado de duas maneiras [1]:

- Declarar elementos da IU em XML. O Android fornece um vocabulário XML direto que corresponde às classes e subclasses de visualização, como as de widgets e layouts. Também é possível usar o Layout Editor do Android Studio para criar o layout XML usando uma interface de arrastar e soltar.

- Instanciar elementos do layout no momento da execução. O aplicativo pode criar objetos `View` e `ViewGroup` (e processar suas propriedades)

programaticamente.

Ao declarar a IU no XML, é possível separar a apresentação do seu aplicativo do código que controla o comportamento dele. O uso de arquivos XML também facilita conseguir layouts diferentes para diferentes orientações e tamanhos de tela [1].

Os atributos do layout XML chamados `layout_something` definem parâmetros de layout para o View apropriados para o `ViewGroup` em que reside. Todos os grupos de visualizações contêm largura e altura (`layout_width` e `layout_height`) e cada visualização é obrigatória para defini-las [1].

É possível especificar largura e altura com medidas exatas, embora não seja recomendável na maioria dos casos. Em geral, usa-se uma destas constantes para definir a largura e a altura [1]:

- `wrap_content` instrui a exibição a redimensionar de acordo com as dimensões exigidas pelo conteúdo.
- `match_parent` instrui sua visualização a assumir o maior tamanho permitido pelo grupo de visualizações parental.

Em geral, a especificação de largura e altura de um layout com unidades absolutas, como pixels, não é recomendada. Em vez disso, o uso de medidas relativas como unidades de pixel de densidade independente (dp), `wrap_content` ou `match_parent` é uma abordagem melhor, porque ajuda a garantir que o app exiba o conteúdo adequadamente nos diversos tamanhos de tela de dispositivos [1].

📕 LinearLayout

O `LinearLayout` organiza os elementos filhos horizontalmente em uma única linha ou verticalmente em uma única coluna. O atributo `android:orientation` define se os elementos serão exibidos na horizontal ou vertical. Por padrão, o `LinearLayout` organiza os elementos na horizontal [2].

```
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:paddingStart="16dp"
   android:paddingEnd="16dp"
   android:orientation="horizontal">

   <!-- Aqui vem os outros elementos filhos -->

 </LinearLayout>
```

📙 FrameLayout

O `FrameLayout` organiza os elementos filhos empilhados um em cima do outro. Por ser difícil organizar a posição dos elementos filhos, usamos este layout para exibir uma única `View` ou para exibir uma `View` sobreposta as outras e realizar sua visualização via código [3].

```
<FrameLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent">

   <!-- Aqui vem os outros elementos filhos -->

 </FrameLayout>
```

📗 ConstraintLayout

O `ConstraintLayout` permite que você crie layouts grandes e complexos com uma hierarquia de visualização plana (sem grupos de visualização aninhados). Todo o poder do `ConstraintLayout` está disponível diretamente nas ferramentas visuais do Layout Editor [4].

Para definir a posição de uma visualização no `ConstraintLayout`, adicione pelo menos uma restrição horizontal e uma vertical para a visualização. Cada restrição representa uma conexão ou alinhamento para outra visualização, o layout pai ou uma linha guia invisível. Cada restrição define a posição da visualização ao longo do eixo vertical ou horizontal. Assim, cada visualização precisa ter pelo menos uma restrição para cada eixo, mas muitas vezes são necessárias mais [4].

Quando você solta uma visualização no Layout Editor, ela permanece onde você a deixou mesmo que não haja restrições. No entanto, isso serve apenas para facilitar a edição. Se uma visualização não tiver restrições quando você executar o layout em um dispositivo, ela será desenhada na posição [0,0], isto é, no canto superior esquerdo [4].

Na Figura 2, o layout fica bem no editor, mas não há restrição vertical na visualização C. Quando esse layout é colocado em um dispositivo, a visualização C fica alinhada horizontalmente às extremidades esquerda e direita da visualização da propriedade A, mas aparece na parte superior da tela, porque não tem restrição vertical. Embora a falta de uma restrição não cause um erro de compilação, o Layout Editor indica a ausência de restrições como um erro na barra de ferramentas. Na Figura 3, este problema é corrigido. [4].

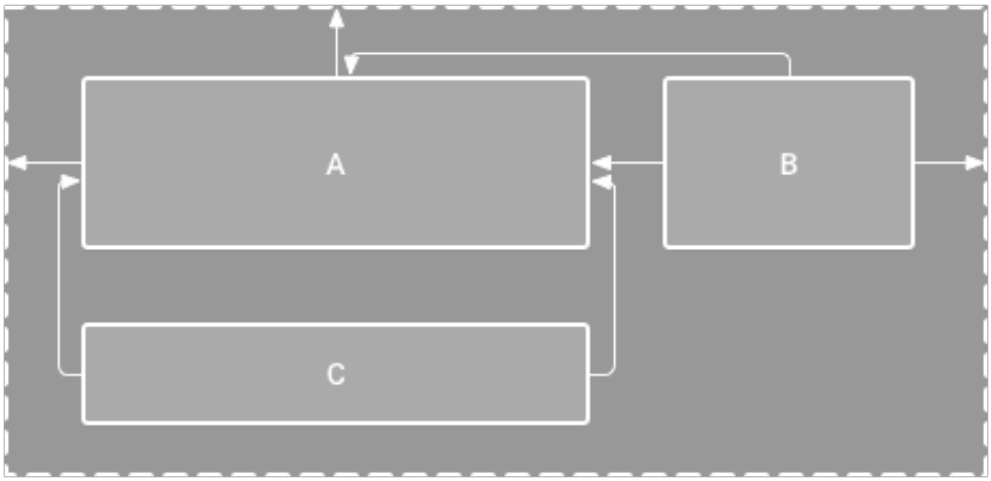

Figura 2: O editor mostra a visualização C abaixo de A, mas não tem restrição vertical.

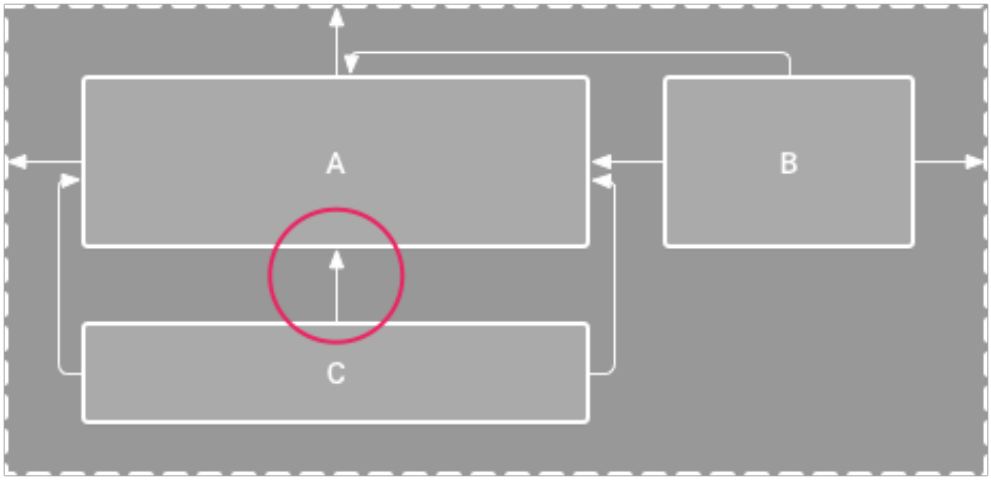

Figura 3: A visualização C está agora verticalmente restrita abaixo da visualização A.

Você pode usar restrições para atingir diferentes tipos de comportamento de layout, conforme descrito nas seções a seguir [4].

Posição do pai

Restringir o lado de uma visualização à borda correspondente do layout. Na Figura 4, o lado esquerdo da visualização está conectado à borda esquerda do layout pai. Você pode definir a distância a partir da borda com margem [4].

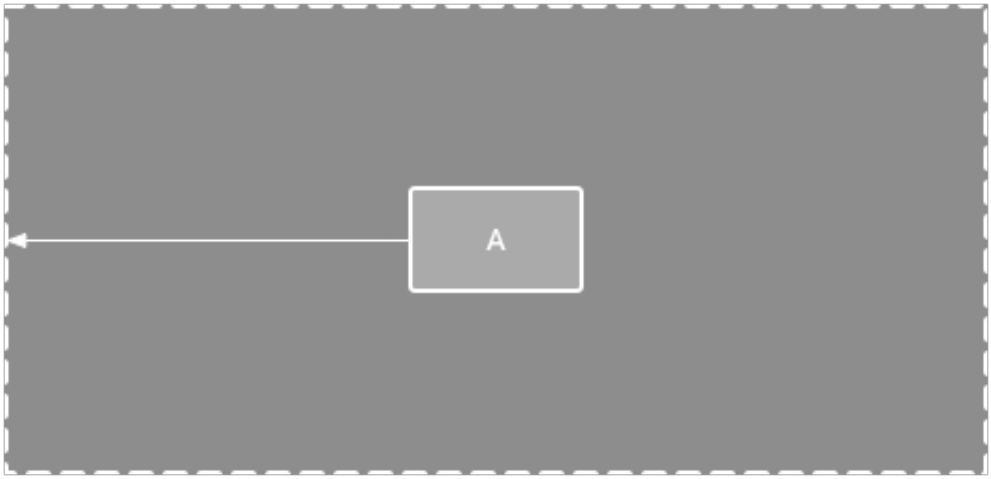

Figura 4: Uma restrição horizontal em relação ao pai.

Posição da ordem

Defina a ordem de exibição de duas visualizações, vertical ou horizontalmente. Na Figura 4, B está restrito a ficar sempre à direita de A, e C está restrito abaixo de A. No entanto, essas restrições não implicam alinhamento, de modo que B ainda possa se mover para cima e para baixo [4].

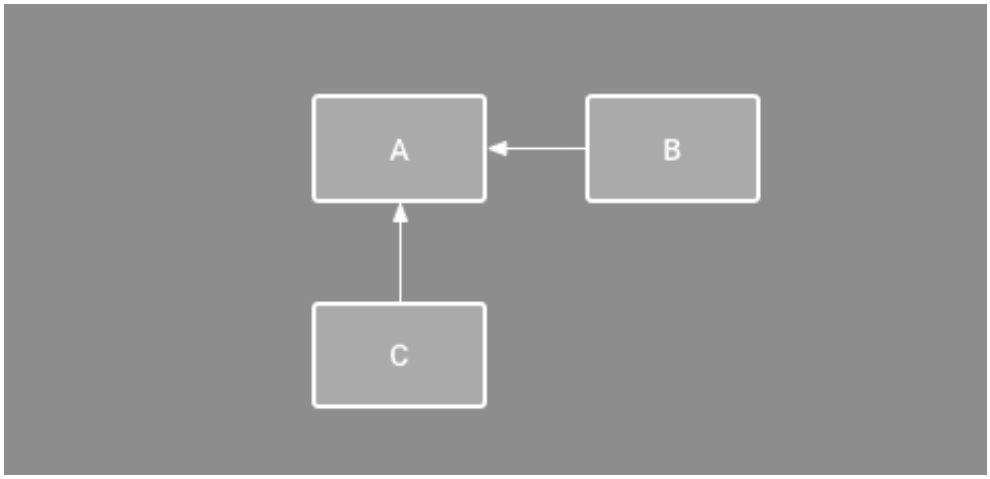

Figura 5: Uma restrição horizontal e vertical.

Alinhamento

Alinhe a borda de uma visualização com a mesma borda de outra. Na Figura 6, o lado esquerdo de B está alinhado com o lado esquerdo de A. Para alinhar os centros de uma visualização, crie uma restrição em ambos os lados [4].

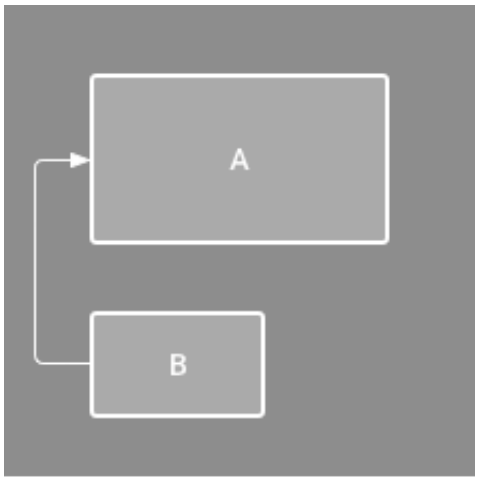

Figura 6: Uma restrição de alinhamento horizontal.

Você pode deslocar o alinhamento arrastando a visualização para dentro a partir da restrição. Por exemplo, a Figura 7 mostra B com um alinhamento de deslocamento de 24 dp. O deslocamento é definido pela margem da visualização restrita [4].

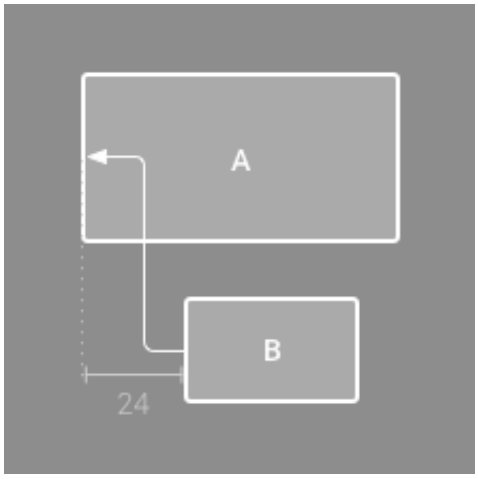

Figura 7: Uma restrição de deslocamento de alinhamento horizontal.

Alinhamento da linha de base

Alinhe a linha de base do texto de uma visualização à linha de base do texto de outra. Na Figura 8, a primeira linha de B está alinhada com o texto em A [4].

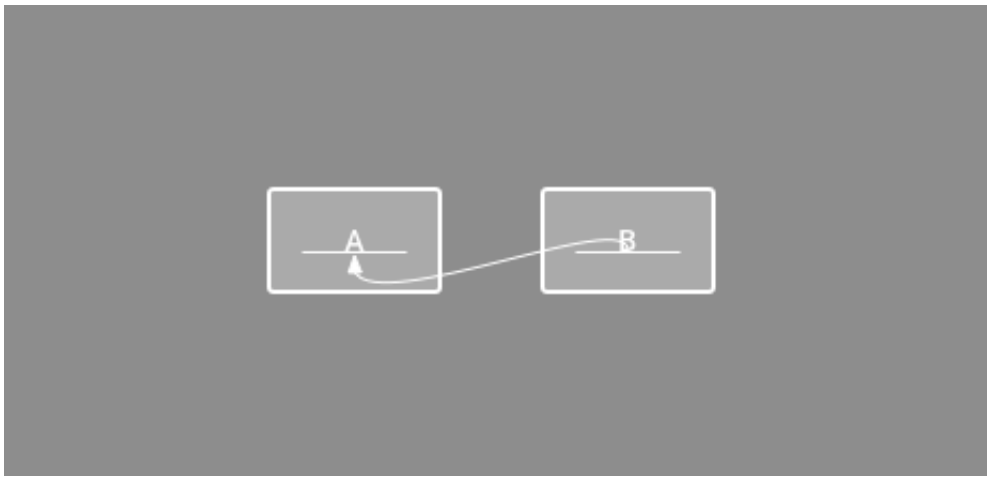

Figura 8: Uma restrição de alinhamento da linha de base.

📘 CardView

Muitas vezes, os apps precisam exibir dados em contêineres com estilo semelhante. Esses contêineres são frequentemente usados em listas para armazenar as informações de cada item. O sistema fornece a API `CardView` como uma maneira fácil de mostrar informações dentro de cards que têm uma aparência consistente em toda a plataforma. Esses cards têm uma elevação padrão acima da visualização em grupo que os contém, fazendo com que o sistema exiba sombras abaixo deles. Os cards são uma maneira fácil de conter um grupo de visualizações e, ao mesmo tempo, fornecer um estilo consistente para o contêiner [5].

Para usar a `CardView`, adicione-a ao arquivo de layout. Use-o como uma visualização em grupo para conter outras visualizações [5].

```
<androidx.cardview.widget.CardView
    xmlns:card="http://schemas.android.com/apk/res-auto"
    android:id="@+id/card_view"
    android:layout_gravity="center"
    android:layout_width="200dp"
    android:layout_height="200dp"
    card:cardCornerRadius="4dp">

    ...

</androidx.cardview.widget.CardView>
```

Os cards são exibidos na tela com uma elevação padrão, fazendo com que o sistema exiba uma sombra abaixo deles. Você pode fornecer uma elevação personalizada para um card com o atributo `card_view:cardElevation`. Isso criará uma sombra mais pronunciada com uma elevação maior, enquanto uma elevação mais baixa resultará em uma sombra mais clara. `CardView` usa a elevação real e sombras dinâmicas no Android 5.0 (API de nível 21) e versões mais recentes, mas usa uma implementação de sombra programática em versões anteriores [5].

O `CardView` possui algumas propriedades de personalização [5]:

- Para definir o raio do canto nos layouts, use o atributo `card:cardCornerRadius`.
- Para definir a cor do plano de fundo de um card, use o atributo `card:cardBackgroundColor`.

📕 ScrollView

O `ScrollView` permite que o elemento filho possa ser ser rolado pela tela. O `ScrollView` aceita apenas um elemento filho. Para adicionar múltiplos elementos é necessário que o filho seja um `ViewGroup` como um `LinearLayout` ou `ConstraintLayout` [6].

O `ScrollView` permite apenas a rolagem vertical. Para implementar a rolagem horizontal é necessário utilizar a classe `HorizontalScrollView` [5].

```
<ScrollView
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent">

   <!-- Aqui vem o elemento filho -->

</ScrollView>
```

> ⚠️ Nunca adicione um `RecyclerView` ou `ListView` dentro de um `ScrollView`. Isso gera problemas de desempenho e de utilização para o usuário [5].

📚 Referência bibliográficas

[1] Layout. **Android Developers**. Acessado em 16/01/2021.

[2] LinearLayout. **Android Developers**. Acessado em 16/01/2021.

[3] LinearLayout. **Android Developers**. Acessado em 16/01/2021.

[4] ConstraintLayout. **Android Developers**. Acessado em 14/01/2021.

[5] CardView. **Android Developers**. Acessado em 14/01/2021.

[6] ScrollView. **Android Developers**. Acessado em 14/01/2021.